

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ



**BÁO CÁO MÔ PHỎNG VÀ KIỂM THỬ
THIẾT KẾ GAME SNAKE TRÊN FPGA**

Đề tài: Thiết kế và kiểm thử hệ thống phần cứng trò chơi Snake điều khiển dải LED WS2812B trên kit FPGA Cyclone II

Sinh viên thực hiện: Trần Hoàng An
MSSV: 2410040
Khóa / Lớp: Khóa 24 / DD24ITN
Chuyên ngành: Thiết kế Vi mạch (IC Design)
GitHub Repository: <https://github.com/aokiii131>

Thành phố Hồ Chí Minh, tháng 09 năm 2026

Mục lục

1	Môi trường Kiểm thử & Quy ước Thời gian	2
1.1	Công cụ Mô phỏng	2
1.2	Quy ước Xung nhịp và Đơn vị Thời gian	2
2	Kết quả Mô phỏng Module WS2812B Driver	3
2.1	Mục tiêu Kiểm thử và Cấu hình Testbench	3
2.2	Kế hoạch Kiểm thử Toàn diện (Verification Test Plan)	3
2.3	Bảng Tổng hợp Trạng thái Kiểm thử	4
2.4	Bảng chứng Mô phỏng Thực tế (Waveform Evidence)	4
2.4.1	Test Case 01: Hành vi Khởi tạo và Reset (Reset Behavior)	4
2.4.2	Test Case 02: Nạp Khung hình vào Bộ đệm (TX Frame Capture)	4
2.4.3	Test Case 03: Quét Tuần tự Pixel và Bit (Serialization Traversal)	5
2.4.4	Test Case 04: Độ chính xác Dạng sóng PWM (Bit Waveform Timing)	6
2.4.5	Test Case 05 & 06: Chuyển Trạng thái Biên và Thời gian Chốt Khung (Boundary Transition & Latch Verification)	7
2.4.6	Test Case 07: Cách ly Khung hình đang truyền (Snapshot Isolation)	8
3	Kết quả Mô phỏng Module Button Conditioner (Chờ bổ sung)	9
4	Kết quả Mô phỏng Module Tick Generator (Chờ bổ sung)	9
5	Kết quả Mô phỏng Module Game Engine	10
5.1	Mục tiêu Kiểm thử và Cấu hình Testbench	10
5.2	Kế hoạch Kiểm thử Toàn diện (Verification Test Plan)	10
5.3	Bảng Tổng hợp Trạng thái Kiểm thử	11
5.4	Bảng chứng Mô phỏng Thực tế (Waveform Evidence)	11
5.4.1	Test Case 01: Khởi tạo và Reset (Reset & Initial State)	11
5.4.2	Test Case 02: Bắt đầu Trò chơi (IDLE → PLAYING)	12
5.4.3	Test Case 03: Động học Di chuyển của Rắn (Snake Movement)	13
5.4.4	Test Case 04: Tạo và Hiển thị Dạn (Bullet Firing & Rendering)	14
5.4.5	Test Case 05: Động học Di chuyển của Dạn (Bullet Movement)	15
5.4.6	Test Case 06: Va chạm Cùng Màu (Same-color Collision)	16
5.4.7	Test Case 07: Va chạm Khác Màu (Different-color Collision)	17
5.4.8	Test Case 08: Kích hoạt Điều kiện Thắng (WIN Condition)	18
5.4.9	Test Case 09: Kích hoạt Điều kiện Thua (LOSE Condition)	19
5.4.10	Test Case 10: Khởi động lại từ Trạng thái Thua (Restart from LOSE)	20
5.4.11	Test Case 11: Xung Kích hoạt Đồng thời (Simultaneous Ticks)	21
5.4.12	Test Case 12: Chống Bắn Dạn Dồn Dập (Rapid Fire Prevention Stress Test)	23
6	Kết quả Mô phỏng Tích hợp Toàn hệ thống Top Module	25
7	Kết quả Mô phỏng Tích hợp Toàn hệ thống (Top-Level Verification)	25
7.1	Mục tiêu Kiểm thử	25
7.2	Quy trình Kiểm thử Đầu-cuối (End-to-End Verification Flow)	25
7.3	Bảng chứng Mô phỏng Dạng sóng (Waveform Evidence)	26
7.3.1	Hành vi Reset, Khởi động và Kích hoạt Dạn xuyên suốt	26
7.3.2	Tình tiến Tự động và Kiểm tra Giao thức PWM tại led_data_out	26
8	Tổng kết Kết quả Kiểm thử	27

8.1	Tổng hợp Kết quả Kiểm thử	27
8.2	Nhận xét Đánh giá	27
9	Kết luận và Hướng Phát triển	27
9.1	Những Kết quả Đạt được	27
9.2	Giới hạn của Báo cáo Kiểm thử	28
9.3	Hướng Phát triển Tiếp theo	28

1 Môi trường Kiểm thử & Quy ước Thời gian

1.1 Công cụ Mô phỏng

- **Phần mềm mô phỏng:** QuestaSim 23.1
- **Ngôn ngữ thiết kế & kiểm thử:** SystemVerilog (IEEE 1800-2012)
- **FPGA Target:** Altera Cyclone II (Kit DE2)

1.2 Quy ước Xung nhịp và Đơn vị Thời gian

- Xung nhịp hệ thống (clk): 50 MHz $\implies T_{clk} = 20$ ns.
- Độ phân giải thời gian mô phỏng: `timescale 1ns/1ps`.

2 Kết quả Mô phỏng Module WS2812B Driver

2.1 Mục tiêu Kiểm thử và Cấu hình Testbench

Module `ws2812b_driver` đảm nhiệm việc chốt dữ liệu hình ảnh logic từ Game Engine và điều chế thành chuỗi xung số NZR đơn dây theo đúng chuẩn thời gian của chip WS2812B.

Để tối ưu hóa thời gian chạy mô phỏng trên ModelSim/Questasim (tránh phải chờ hàng triệu chu kỳ xung nhịp cho 60 LED và $300\mu s$ Latch), môi trường Testbench (`ws2812b_driver_tb`) đã override các tham số phần cứng thông qua cơ chế Parameterization:

- `LED_COUNT = TEST_LEDS = 3` (Kiểm thử rút gọn trên 3 LED thay vì 60 LED).
- `RES = TEST_RES = 50` (Rút ngắn thời gian chốt khung hình xuống 50 chu kỳ clock thay vì 15,000 chu kỳ).
- Chu kỳ Master Clock: $T_{clk} = 20\text{ ns}$ (50 MHz).
- Tham số định thời bit chuẩn: $T_{0H} = 15\text{ ck}$ (300 ns), $T_{0L} = 40\text{ ck}$ (800 ns), $T_{1H} = 40\text{ ck}$ (800 ns), $T_{1L} = 15\text{ ck}$ (300 ns). Tổng chu kỳ mỗi bit là 55 ck (1100 ns).

2.2 Kế hoạch Kiểm thử Toàn diện (Verification Test Plan)

Kế hoạch kiểm thử của module bao gồm 7 kịch bản kiểm tra toàn diện hoạt động của mạch, từ phản hồi mức thanh ghi, định thời dạng sóng PWM, chuyển trạng thái biên cho tới tính độc lập của bộ đệm khung hình:

1. **TC01 - Reset Behavior:** Kiểm tra tín hiệu `rst` khởi tạo thành công FSM về trạng thái `LOAD`, các thanh ghi đếm được đặt về giá trị mặc định (`pixel_counter = 0`, `bit_counter = 23`, `timing_counter = 0`) và ngõ ra `led_data_out` giữ mức logic 0.
2. **TC02 - Frame Capturing (LOAD):** Kiểm tra khả năng chốt dữ liệu mảng từ cổng `pixel_data` vào bộ đệm nội bộ `tx_frame` trong chu kỳ `LOAD` bằng kích thích ngẫu nhiên có ràng buộc `$urandom_range(4, 0)`.
3. **TC03 - Serialization Traversal (SEND):** Kiểm tra quy trình tuần tự hóa dữ liệu khi phát khung: duyệt tuần tự pixel từ $0 \rightarrow 2$ (`TEST_LEDS = 3`) và truyền bit theo thứ tự MSB-first từ $23 \rightarrow 0$ cho từng pixel.
4. **TC04 - Bit Waveform Timing:** Kiểm tra độ chính xác xung PWM trên ngõ ra `led_data_out` theo chuẩn WS2812B: Bit 0 (15 ck HIGH/40 ck LOW) và Bit 1 (40 ck HIGH/15 ck LOW) tổng chu kỳ mỗi bit đạt chính xác 55 ck (1100 ns).
5. **TC05 - Boundary Transition (SEND $\rightarrow$ LATCH):** Xác nhận điều kiện biên tại thời điểm kết thúc khung hình (`pixel_counter = 2`, `bit_counter = 0`, `timing_counter = 54`), FSM chuyển chính xác sang trạng thái `LATCH`.
6. **TC06 - Frame Latch Integrity:** Kiểm tra trong suốt trạng thái `LATCH`, ngõ ra `led_data_out` luôn giữ mức LOW tuyệt đối đủ 50 chu kỳ (1000 ns theo `TEST_RES = 50`) trước khi FSM tự động quay về `LOAD`.
7. **TC07 - Frame Snapshot Isolation:** Kiểm tra tính cô lập khung truyền: thay đổi đột ngột giá trị `pixel_data` sang `{3'b010, 3'b011, 3'b000}` ngay khi FSM đang ở trạng thái `SEND`, đảm bảo bộ đệm `tx_frame` và bus giải mã 24-bit không bị biến động.

2.3 Bảng Tổng hợp Trạng thái Kiểm thử

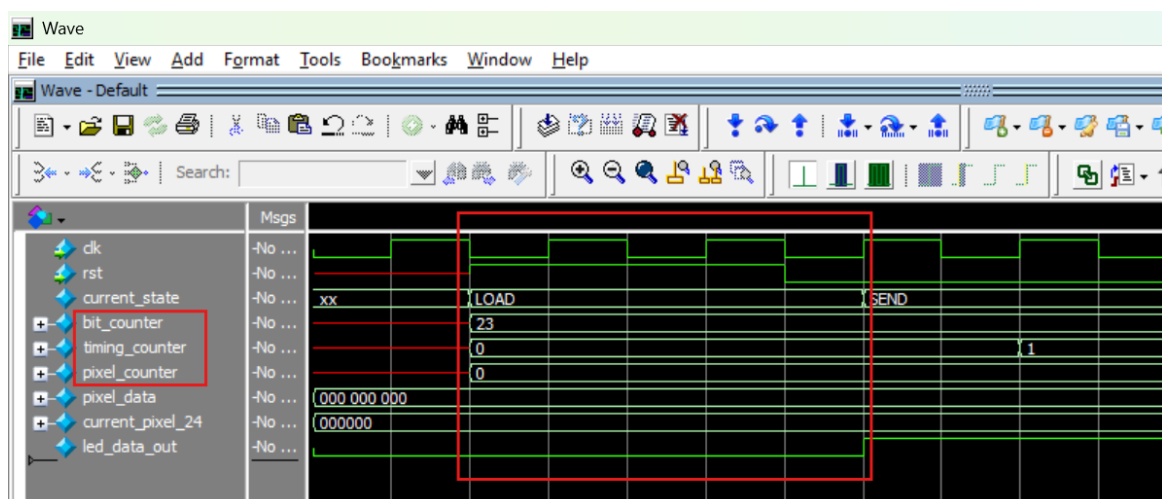
Bảng 1: Bảng tổng hợp kết quả thực thi các Test Cases của WS2812B Driver

STT	Kịch bản	Tiêu chí Đánh giá	Kết quả
TEST CASE 1	Reset Behavior	FSM vào LOAD, các bộ đếm reset về 0, <code>led_data_out = 0</code>	PASS
TEST CASE 2	TX Frame Capture	Dữ liệu <code>tx_frame</code> khớp 100% với <code>pixel_data</code>	PASS
TEST CASE 3	Sending Completion	Quét tuần tự pixel 0 $\rightarrow$ 2, bit 23 $\rightarrow$ 0 và hoàn tất phát chuyển sang LATCH	PASS
TEST CASE 4	Latching Completion	Giữ mức LOW đủ 50 ck ($1\mu s$) và hoàn tất chốt để quay về LOAD	PASS
TEST CASE 5	Snapshot Isolation	Đổi <code>pixel_data</code> giữa lúc SEND không làm biến đổi <code>tx_frame</code> đang phát	PASS

2.4 Bằng chứng Mô phỏng Thực tế (Waveform Evidence)

2.4.1 Test Case 01: Hành vi Khởi tạo và Reset (Reset Behavior)

- **Điều kiện kích thích:** Tác động xung `rst = 1` trong 2 chu kỳ xung nhịp, sau đó hạ về 0 tại cạnh âm của xung nhịp.
- **Kết quả mô phỏng:**
 - FSM khởi tạo tức thì vào trạng thái LOAD .
 - Các biến đếm `pixel_counter = 0`, `bit_counter = 23`, và `timing_counter = 0`.
 - Tín hiệu `led_data_out` giữ mức logic 0, không xuất hiện bất kỳ xung nhiễu nào.



Hình 1: Dạng sóng kiểm tra hành vi Reset của module Driver

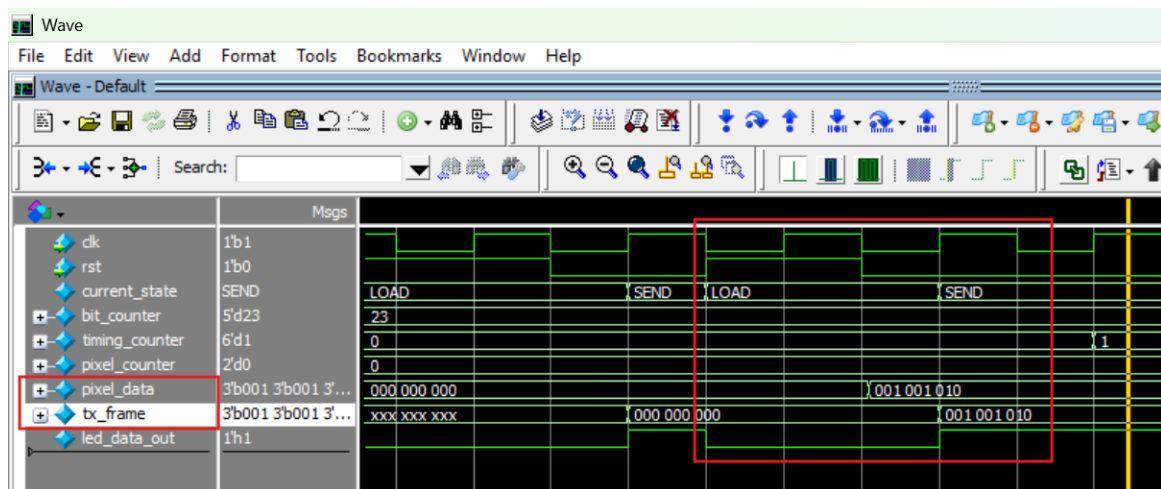
2.4.2 Test Case 02: Nạp Khung hình vào Bộ đệm (TX Frame Capture)

- **Điều kiện kích thích:** Sinh ngẫu nhiên giá trị màu cho 3 pixel thông qua hàm `$urandom_range(4,0)`. Quan sát sự tương quan giữa mảng đầu vào `pixel_data` và mảng thanh ghi nội bộ `tx_frame`.

- **Kết quả Console log:** Toàn bộ giá trị từ `pixel_data[0..2]` được sao chép nguyên vẹn sang `tx_frame[0..2]` ngay khi FSM trải qua trạng thái `LOAD`:

```
# -----
# TEST CASE 2: TX FRAME CAPTURING
# Time = 121000 ns | State = SEND
# pixel_data[0] = 001 | tx_frame[0] = 001
# pixel_data[1] = 001 | tx_frame[1] = 001
# pixel_data[2] = 010 | tx_frame[2] = 010
# ** Note: $finish      : tb/ws2812b_driver_tb.sv(87)
#   Time: 220 ns  Iteration: 1  Instance: /ws2812b_driver_tb
```

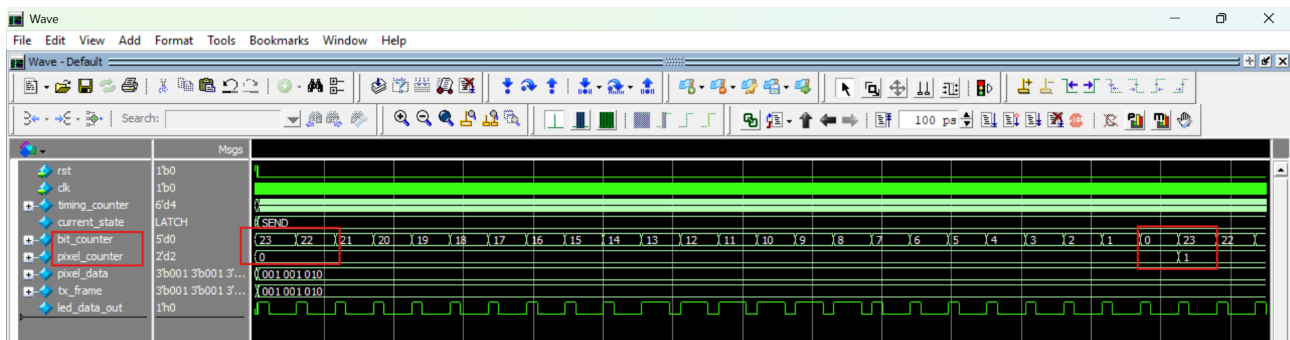
Hình 2: Console Log ghi nhận dữ liệu nạp vào `tx_frame` từ `pixel_data`



Hình 3: Dữ liệu hình ảnh được chốt an toàn vào thanh ghi truyền dẫn

2.4.3 Test Case 03: Quét Tuần tự Pixel và Bit (Serialization Traversal)

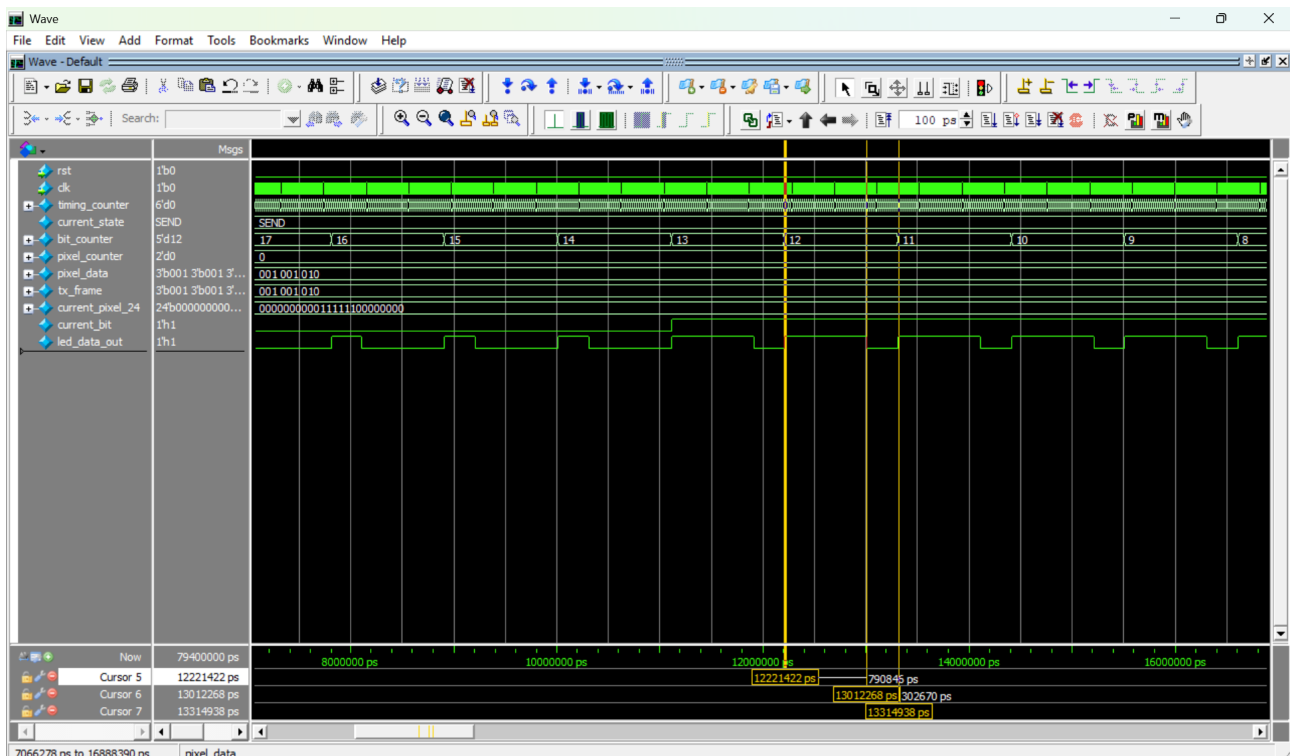
- **Điều kiện kích thích:** Quan sát tiến trình vận hành của FSM khi duy trì ở trạng thái `SEND`.
- **Phân tích dạng sóng:**
 - Bộ đếm `bit_counter` thực hiện đếm lùi chính xác từ 23 về 0 cho mỗi gói dữ liệu pixel.
 - Ngay tại chu kỳ `bit_counter` đạt giá trị 0 và hoàn tất 55 chu kỳ truyền, `pixel_counter` tự động tăng thêm 1 đơn vị (từ 0 lên 1) đồng thời `bit_counter` lập tức nạp lại giá trị 23 để bắt đầu truyền pixel tiếp theo.
 - Cơ chế này đảm bảo toàn bộ khung hình được tuần tự hóa liên tục theo đúng chuẩn MSB-first mà không bị mất nhịp.



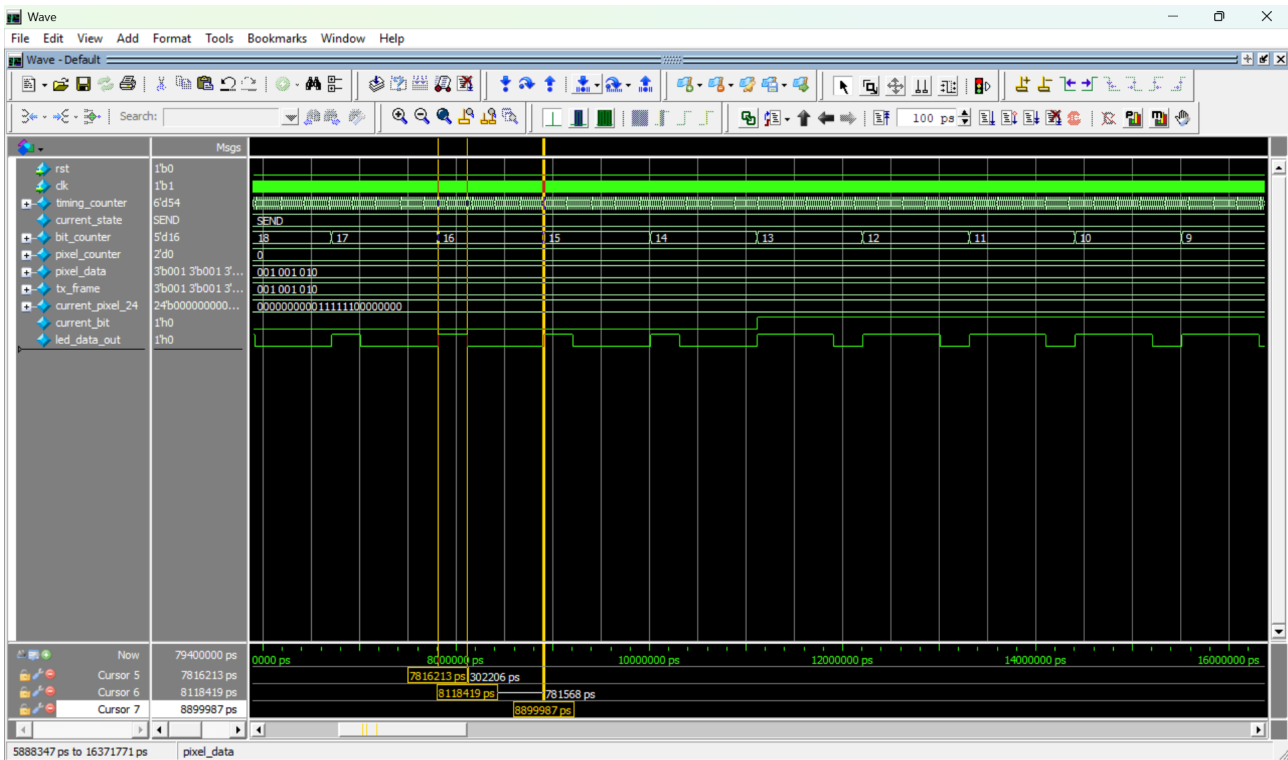
Hình 4: Dạng sóng tiến trình quét tuần tự Pixel và Bit trong trạng thái SEND

2.4.4 Test Case 04: Độ chính xác Dạng sóng PWM (Bit Waveform Timing)

- **Điều kiện kích thích:** Sử dụng công cụ đo lường (Cursor) trên ModelSim để xác minh độ rộng xung HIGH và LOW của tín hiệu ngõ ra `led_data_out` khi phát bit 1 và bit 0.
- **Kết quả đo đạc:**
 - **Mã hóa Bit 1:** Quan sát tại thời điểm `bit_counter = 12`, tín hiệu `current_bit` nhận giá trị 1 từ bus `current_pixel_24`. Dạng sóng ngõ ra `led_data_out` ghi nhận mức HIGH xấp xỉ 800 ns (T_{1H}) và mức LOW xấp xỉ 300 ns (T_{1L}). Tổng thời lượng đạt chính xác 55 chu kỳ (1100 ns).
 - **Mã hóa Bit 0:** Khi `current_bit` mang giá trị 0, tín hiệu duy trì mức HIGH trong 15 chu kỳ xung nhịp (300 ns) và mức LOW trong 40 chu kỳ (800 ns). Tổng thời lượng đạt chính xác 55 chu kỳ (1100 ns).



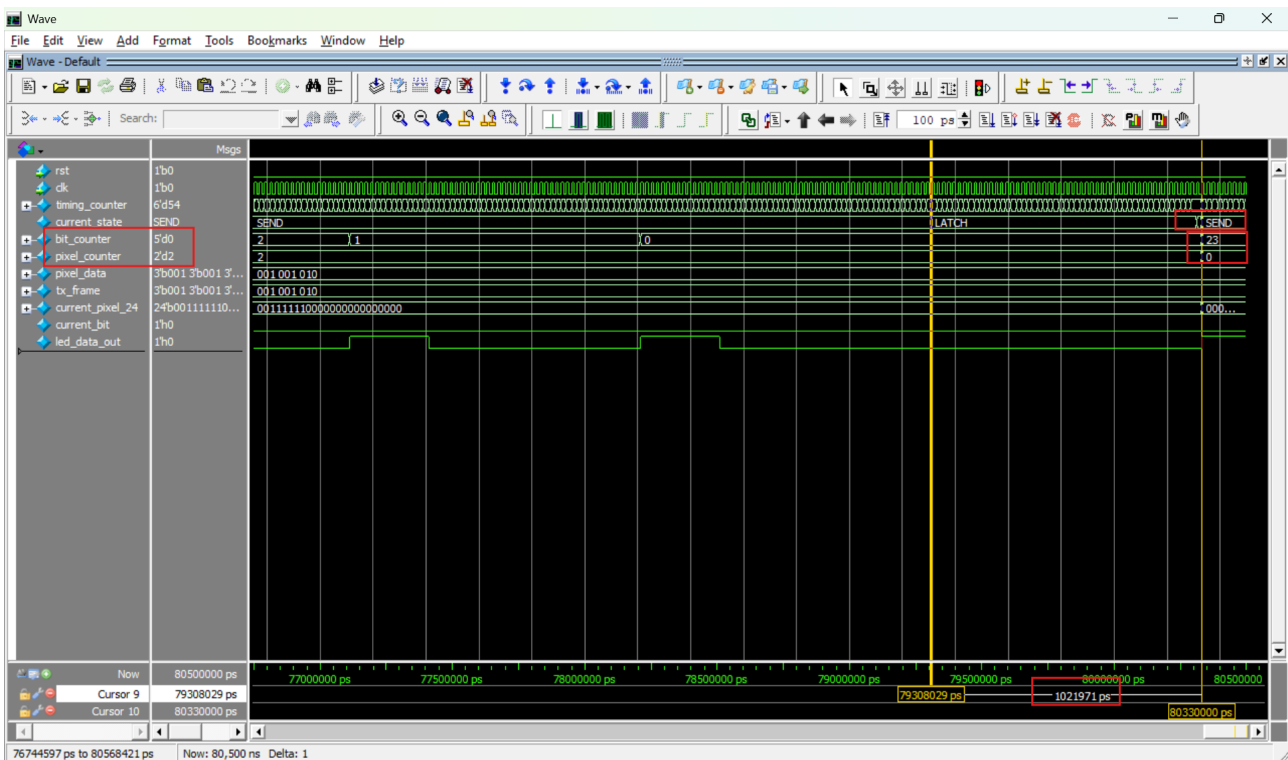
Hình 5: Phân tích độ rộng xung PWM khi truyền Bit 1 (tại bit counter = 12)



Hình 6: Phân tích độ rộng xung PWM khi truyền Bit 0

2.4.5 Test Case 05 & 06: Chuyển Trạng thái Biên và Thời gian Chốt Khung (Boundary Transition & Latch Verification)

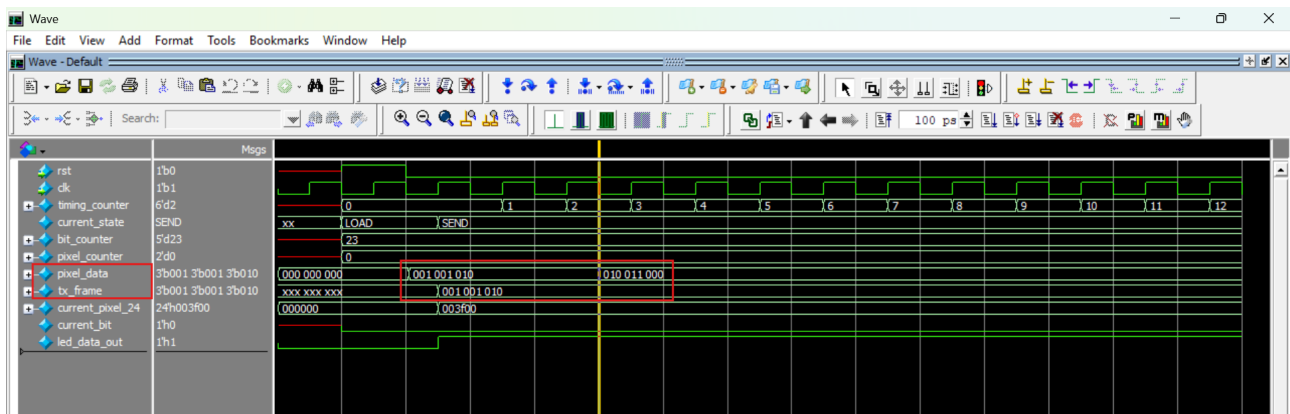
- **Điều kiện kích thích:** Quan sát thời điểm hoàn tất truyền bit cuối cùng của pixel cuối cùng trong khung hình (`pixel_counter = 2`, `bit_counter = 0`) và tiến hành thực thi trạng thái LATCH.
- **Phân tích dạng sóng:**
 - **Chuyển trạng thái biên (Boundary Transition):** Dúng tại thời điểm `pixel_counter = 2` (pixel thứ 3 theo tham số `TEST_LEDS = 3`) và `bit_counter = 0` kết thúc 55 chu kỳ truyền xung, FSM chuyển trạng thái từ SEND sang LATCH tại vị trí vạch đo Cursor 9 ($79.308 \mu s$). Sau đó, khi RES hết, FSM tự động quay về trạng thái LOAD tại vạch đo Cursor 10 ($80.330 \mu s$).
 - **Tính toàn vẹn thời gian Latch (Latch Integrity):** Trong suốt trạng thái LATCH, đường tín hiệu ngõ ra `led_data_out` được giữ ở mức 0 tuyệt đối.
 - **Đo kiểm chu kỳ trễ RES:** Theo cấu hình Testbench với `TEST_RES = 50`, thời gian lý thuyết giữ mức LOW là $50 \times 20 \text{ ns} = 1000 \text{ ns}$ ($1.0 \mu s$). Kết quả đo thực tế giữa Cursor 9 và Cursor 10 đạt xấp xỉ $1021971 \text{ ps} \approx 1022 \text{ ns}$ ($\approx 1.02 \mu s$), hoàn toàn khớp với chu kỳ thiết kế trước khi hệ thống kết thúc khung truyền.



Hình 7: Dạng sóng chuyển trạng thái SEND sang LATCH và giữ mức LOW trong thời gian RES

2.4.6 Test Case 07: Cách ly Khung hình đang truyền (Snapshot Isolation)

- **Điều kiện kích thích:** Khi FSM đang thực thi trạng thái SEND (tại nhịp `timing_counter` = 2), tín hiệu ngõ vào `pixel_data` bị thay đổi đột ngột từ {001, 001, 010} sang {010, 011, 000}.
- **Phân tích dạng sóng:**
 - Mặc dù ngõ vào `pixel_data` bị cập nhật dữ liệu mới giữa chừng, mảng thanh ghi nội bộ `tx_frame` vẫn giữ nguyên giá trị đã chốt trước đó là {001, 001, 010}.
 - Bus dữ liệu 24-bit `current_pixel_24` tiếp tục duy trì giá trị 24'h003F00 (mã màu RED) tương ứng với phần tử `tx_frame[0]` mà không bị ảnh hưởng bởi biến động đầu vào.
 - Kết quả chứng minh kiến trúc chốt đệm `tx_frame` đã thực hiện cô lập hoàn toàn khung truyền hình ảnh đang phát ra LED khỏi các cập nhật logic tức thời từ Game Engine, ngăn ngừa triệt để hiện tượng xé hình (frame tearing).



Hình 8: Dạng sóng kiểm tra tính cô lập dữ liệu: pixel_data thay đổi giữa chừng nhưng tx_frame và current_pixel_24 không bị ảnh hưởng

- 3 Kết quả Mô phỏng Module Button Conditioner (Chờ bổ sung)
- 4 Kết quả Mô phỏng Module Tick Generator (Chờ bổ sung)

5 Kết quả Mô phỏng Module Game Engine

5.1 Mục tiêu Kiểm thử và Cấu hình Testbench

Module `game_engine` là khối xử lý trung tâm của hệ thống, đảm nhiệm quản lý máy trạng thái trò chơi (FSM), cập nhật tọa độ rắn (*snake*), đạn (*bullet*), giải quyết va chạm (*collision*) và liên tục xuất mảng `pixel_data` cho khối hiển thị.

Để tối ưu hóa thời gian mô phỏng trên ModelSim/QuestaSim (tránh phải chờ hàng triệu chu kỳ xung nhịp cho các chu kỳ thực tế 50 ms và 250 ms), môi trường kiểm thử (`game_engine_tb`) áp dụng cơ chế điều khiển xung tick mô phỏng rút gọn:

- **Chu kỳ Master Clock:** $T_{clk} = 20 \text{ ns}$ (50 MHz).
- **Độ phân giải thời gian mô phỏng:** `timescale 1ns/1ps`.
- **Cấu hình dải LED:** `LED_COUNT = 60 pixels`, `DEFAULT_LENGTH = 5 segments`.
- **Rút ngắn Timing bằng Simulation Ticks:** Thay vì chờ bộ đếm 2.5×10^6 chu kỳ (50 ms cho `bullet_tick`) và 12.5×10^6 chu kỳ (250 ms cho `snake_tick`), testbench trực tiếp tạo các xung tích cực mức cao kéo dài đúng 1 chu kỳ clock theo kịch bản để kích hoạt cập nhật động lực học trò chơi.

5.2 Kế hoạch Kiểm thử Toàn diện (Verification Test Plan)

Kế hoạch kiểm thử tập trung xác minh toàn bộ các quy tắc trò chơi, datapath tính toán look-ahead và máy trạng thái FSM thông qua 12 kịch bản:

1. **TC01 – Reset & Initial State:** Kiểm tra tín hiệu reset đưa FSM về IDLE, rắn dài 5 đốt tại vị trí 55 $\rightarrow$ 59 với đúng chuỗi màu, đạn ở trạng thái không tích cực (`bullet_active = 0`).
2. **TC02 – IDLE $\rightarrow$ PLAYING:** Kiểm tra sự kiện nút nhấn đầu tiên kích hoạt chuyển trạng thái FSM sang PLAYING mà không bắn đạn ngay lập tức.
3. **TC03 – Snake Movement:** Kiểm tra tọa độ đầu rắn `head_position` giảm 1 đơn vị và toàn bộ mảng `snake_array` dịch chuyển chuẩn xác mỗi khi nhận xung `snake_tick`.
4. **TC04 – Bullet Firing:** Kiểm tra sự kiện nhấn phím trong trạng thái PLAYING sinh đạn tại vị trí LED 0 với mã màu tương ứng (Red, Green, Blue, Yellow).
5. **TC05 – Bullet Movement:** Kiểm tra tọa độ đạn `bullet_position` tăng dần từng bước theo mỗi xung `bullet_tick`.
6. **TC06 – Same-color Collision:** Đạn chạm đầu rắn và trùng màu: đầu rắn bị tiêu biến, chiều dài rắn giảm 1 đơn vị, đạn bị vô hiệu hóa.
7. **TC07 – Different-color Collision:** Đạn chạm đầu rắn nhưng sai màu: chiều dài và màu sắc các đốt rắn giữ nguyên, đạn bị vô hiệu hóa.
8. **TC08 – WIN Condition:** Rắn bị tiêu diệt hoàn toàn (`snake_length = 0`): FSM chuyển sang trạng thái WIN.
9. **TC09 – LOSE Condition:** Đầu rắn chạm mốc LED 0 (`head_position = 0`): FSM chuyển sang trạng thái LOSE.

10. **TC10 – Simultaneous Tick / Look-Ahead:** Kiểm tra tính chính xác của khối Look-ahead khi `snake_tick` và `bullet_tick` kích hoạt đồng thời, chống hiện tượng xuyên đạn (*Anti-Tunneling*).
11. **TC11 – Multiple Fire Prevention:** Khi đạn đang bay (`bullet_active = 1`), mọi xung `btn_event` bổ sung đều bị bỏ qua, không sinh thêm đạn.
12. **TC12 – Terminal State Behavior:** Tại trạng thái WIN hoặc LOSE, mọi chuyển động dừng hẳn và toàn dải `pixel_data` render màu cố định (Solid Green cho WIN, Solid Red cho LOSE).

5.3 Bảng Tổng hợp Trạng thái Kiểm thử

Bảng 2: Bảng tổng hợp kết quả thực thi các Test Cases của Game Engine

STT	Kịch bản Kiểm thử	Tiêu chí Đánh giá	Kết quả
TC01	Reset & Initial State	FSM vào IDLE; Rắn dài 5 đốt (55–59); Đạn inactive	PASS
TC02	IDLE → PLAYING	Xung nút đầu tiên chuyển FSM sang PLAYING; Chưa sinh đạn	PASS
TC03	Snake Movement	<code>head_position</code> giảm 1; Mảng dịch 1 vị trí khi có <code>snake_tick</code>	PASS
TC04	Bullet Firing	Sinh đạn tại LED 0 đúng màu tương ứng với <code>btn_event[3:0]</code>	PASS
TC05	Bullet Movement	<code>bullet_position</code> tăng 1 vị trí mỗi khi có <code>bullet_tick</code>	PASS
TC06	Same-color Collision	Trúng đạn cùng màu: <code>snake_length</code> giảm 1; Đạn bị thu hồi	PASS
TC07	Different-color Collision	Sai màu: Thân rắn giữ nguyên; Đạn bị thu hồi	PASS
TC08	WIN Condition	<code>snake_length = 0</code> đưa FSM chuyển sang WIN	PASS
TC09	LOSE Condition	<code>head_position = 0</code> đưa FSM chuyển sang LOSE	PASS
TC10	Simultaneous Tick Look-Ahead	Xử lý va chạm chuẩn xác khi 2 tick đồng thời; Không bị vượt mặt	PASS
TC11	Multiple Fire Prevention	Nhấn nút khi <code>bullet_active = 1</code> không làm ghi đè đạn đang bay	PASS
TC12	Terminal State Behavior	Đóng băng cập nhật logic; Toàn dải xuất màu Solid GREEN / RED	PASS

5.4 Bằng chứng Mô phỏng Thực tế (Waveform Evidence)

5.4.1 Test Case 01: Khởi tạo và Reset (Reset & Initial State)

Mục tiêu: Kiểm tra phản hồi của hệ thống khi tín hiệu reset bất đồng bộ được kích hoạt, đảm bảo FSM chuyển về trạng thái IDLE, rắn được khởi tạo đúng độ dài và vị trí trên dải LED, đồng thời không có viên đạn nào ở trạng thái kích hoạt.

Điều kiện kích thích:

- Tác động xung reset tích cực mức cao `rst_sw = 1` trong 2 chu kỳ xung nhịp.
- Hạ `rst_sw` về 0 tại cạnh âm của xung nhịp.
- Không tác động bất kỳ sự kiện nút nhấn nào (`btn_event = 4'b0000`).

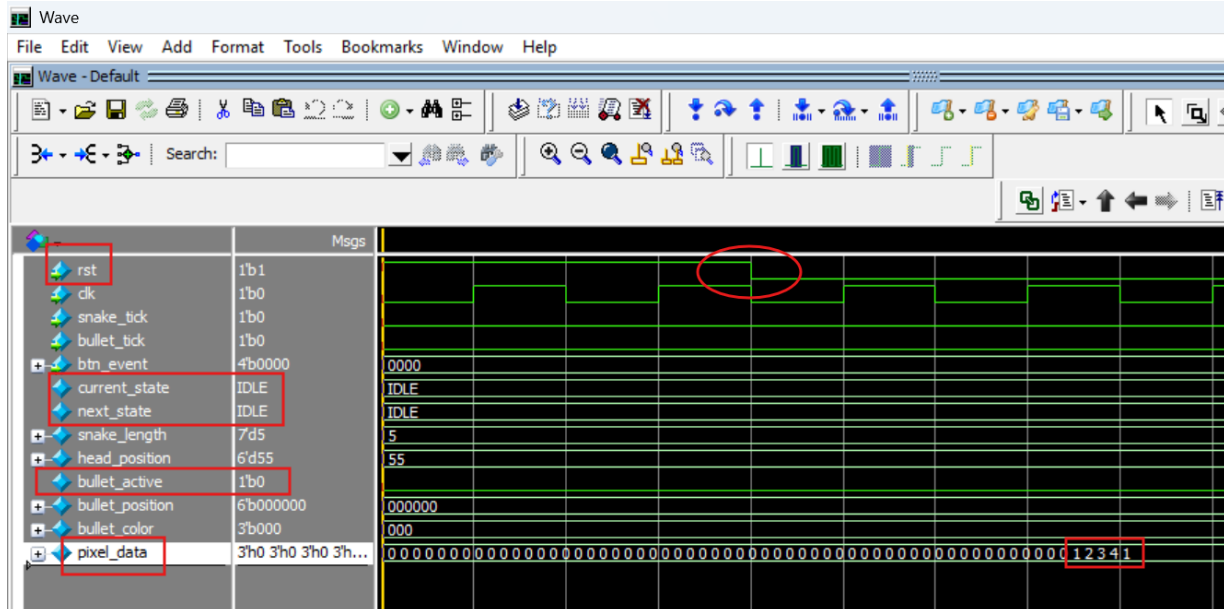
Kết quả mong đợi:

- FSM chuyển về trạng thái IDLE (`current_state = 2'b00`).
- Độ dài rắn `snake_length` khởi tạo bằng 5.
- Vị trí đầu rắn `head_position` khởi tạo tại LED 55.
- Các phần tử mảng thân rắn từ vị trí 55 đến 59 nạp đúng chuỗi màu mặc định: RED, GREEN, BLUE, YELLOW, RED.
- Trạng thái đạn không kích hoạt (`bullet_active = 0`), `bullet_position = 0`.

Kết quả mô phỏng:

Hình 9 minh họa dạng sóng khi tác động tín hiệu reset lên module `game_engine`. Ngay khi `rst_sw` lên mức cao, FSM lập tức chuyển về trạng thái IDLE. Các thanh ghi cấu hình thân rắn được nạp chính xác với `head_position = 55` và `snake_length = 5`, các LED từ 55 đến 59 nhận các mã màu tương ứng. Đồng thời, cờ `bullet_active` duy trì ở mức 0 và không có viên đạn nào xuất hiện trên dải LED.

Kết luận: Test Case 01 đạt yêu cầu (PASS).



Hình 9: Dạng sóng kiểm tra hành vi Reset và khởi tạo của Game Engine

5.4.2 Test Case 02: Bắt đầu Trò chơi (IDLE → PLAYING)

Mục tiêu: Kiểm tra khả năng nhận diện sự kiện bắt đầu game của FSM khi có tín hiệu nhấn nút từ người chơi, đảm bảo hệ thống chuyển đúng từ trạng thái IDLE sang PLAYING mà không sinh đạn ngay trong chu kỳ đầu tiên.

Điều kiện kích thích:

- Hệ thống đang duy trì ổn định ở trạng thái IDLE sau reset.
- Đưa một xung sự kiện nút nhấn `btn_event[3] = 1` (phím màu YELLOW) kéo dài đúng 1 chu kỳ xung nhịp.
- Các tín hiệu `snake_tick` và `bullet_tick` chưa được kích hoạt.

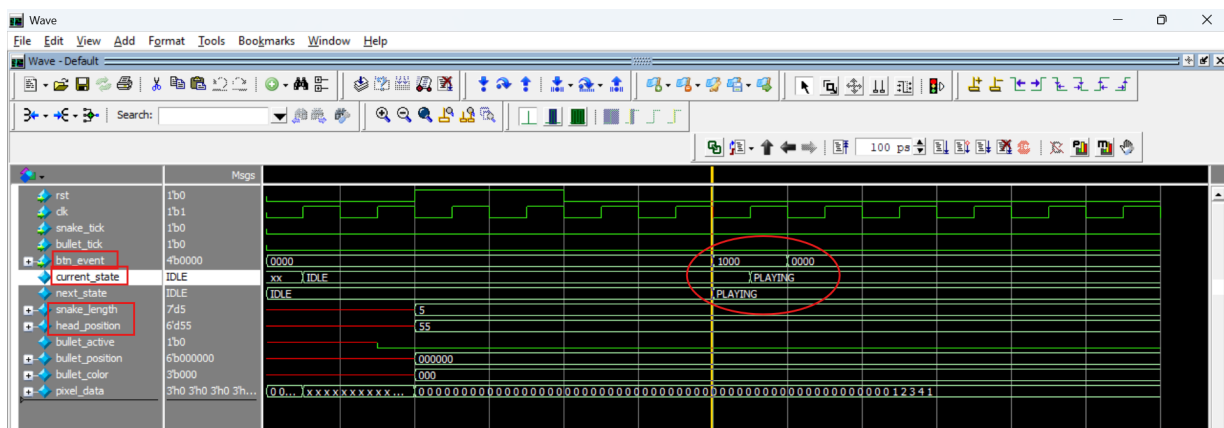
Kết quả mong đợi:

- FSM chuyển trạng thái từ IDLE sang PLAYING tại chu kỳ xung nhịp tiếp theo.
- Tín hiệu `bullet_active` vẫn duy trì ở mức 0, chứng minh rằng sự kiện nút nhấn ban đầu chỉ có tác dụng khởi động game).
- Tọa độ đầu rắn `head_position` và chiều dài `snake_length` không bị thay đổi.

Kết quả mô phỏng:

Hình 10 mô tả đáp ứng của `game_engine` khi nhận xung kích hoạt trò chơi. Ngay tại cạnh lên xung nhịp sau khi `btn_event[3]` kích hoạt, biến trạng thái `current_state` chuyển đổi chính xác sang PLAYING. Cờ `bullet_active` tiếp tục giữ mức logic 0, chứng minh rằng sự kiện nút nhấn đầu tiên đã được FSM lọc đúng chức năng khởi động mà không gây hiện tượng bắn đạn ngoài ý muốn.

Kết luận: Test Case 02 đạt yêu cầu (PASS).



Hình 10: Dạng sóng kiểm tra chuyển trạng thái từ IDLE sang PLAYING

5.4.3 Test Case 03: Động học Di chuyển của Rắn (Snake Movement)

Mục tiêu: Kiểm tra phản hồi của thân rắn khi có xung kích hoạt `snake_tick`, đảm bảo vị trí đầu rắn `head_position` giảm dần về phía LED 0 và toàn bộ mảng `snake_array` được dịch chuyển chính xác theo nguyên lý thanh ghi dịch.

Điều kiện kích thích:

- Hệ thống đang hoạt động ở trạng thái PLAYING.
- Đưa một xung `snake_tick = 1` kéo dài đúng 1 chu kỳ xung nhịp.
- Tín hiệu `bullet_tick = 0` và không có sự kiện nút nhấn (`btn_event = 4'b0000`).

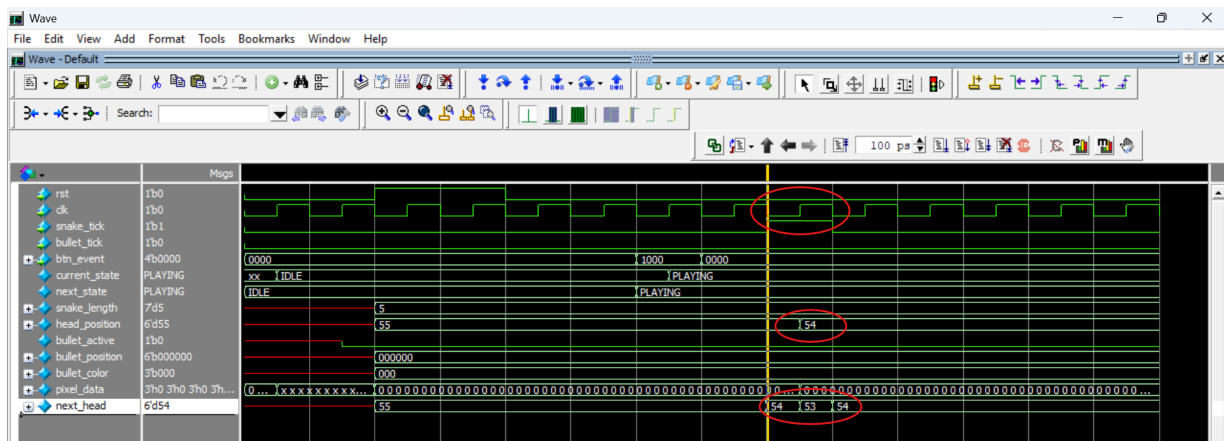
Kết quả mong đợi:

- Tọa độ `head_position` giảm đúng 1 đơn vị (từ 55 xuống 54) tại cạnh lên xung nhịp tiếp theo.
- Các phần tử trong mảng thân rắn thực hiện dịch chuyển sang trái (chỉ số thấp hơn) 1 vị trí.
- Chiều dài rắn `snake_length` được bảo toàn nguyên vẹn (bằng 5).
- Thứ tự màu của các đốt thân rắn không bị sai lệch hay mất mát dữ liệu.

Kết quả mô phỏng:

Hình 11 thể hiện dạng sóng di chuyển của rắn dưới tác động của xung định thời. Khi xuất hiện xung `snake_tick`, bộ tính toán look-ahead cập nhật `next_head = 54`. Tại cạnh lên tiếp theo của `clk`, thanh ghi `head_position` cập nhật giá trị mới là 54, đồng thời mảng thanh ghi dịch thực hiện tịnh tiến toàn bộ các mã màu của thân rắn sang vị trí mới tương ứng. Độ dài `snake_length` vẫn giữ nguyên giá trị 5, phản ánh chính xác hành vi di chuyển tự nhiên của rắn.

Kết luận: Test Case 03 đạt yêu cầu (PASS).



Hình 11: Dạng sóng kiểm tra sự tịnh tiến của thân rắn theo xung `snake_tick`

5.4.4 Test Case 04: Tạo và Hiển thị Đạn (Bullet Firing & Rendering)

Mục tiêu: Kiểm tra khả năng khởi tạo đạn khi người chơi nhấn nút trong trạng thái PLAYING và đảm bảo giá trị màu của đạn được ánh xạ lập tức lên mảng `pixel_data` tại vị trí LED 0.

Điều kiện kích thích:

- FSM đang ở trạng thái PLAYING và chưa có đạn hoạt động (`bullet_active = 0`).
- Kích hoạt xung `btn_event = 4'b0001` (phím màu RED) trong đúng 1 chu kỳ clock tại sườn âm.
- Sau 1 chu kỳ, hạ `btn_event` về 4'b0000.

Kết quả mong đợi:

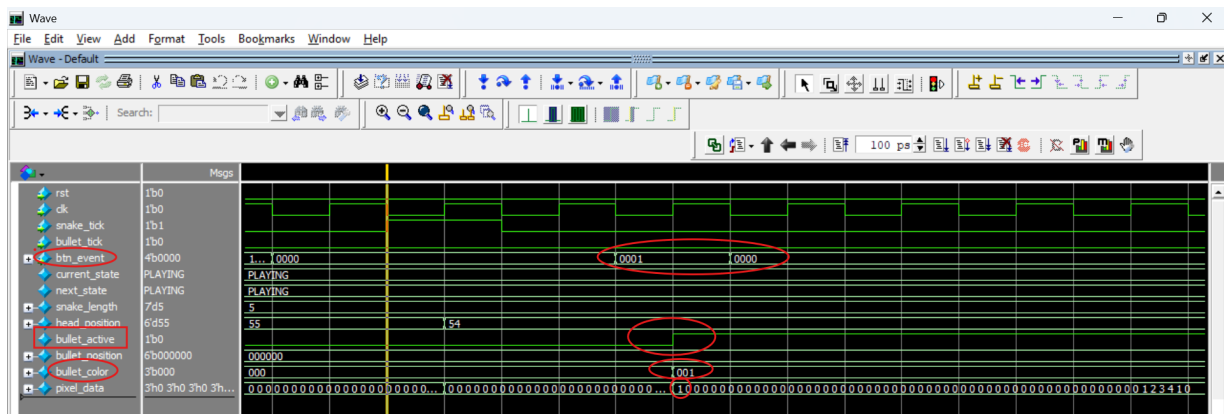
- Tín hiệu `bullet_active` chuyển lên mức 1.
- Tọa độ khởi tạo `bullet_position = 0` và màu đạn `bullet_color = RED (3'b001)`.

- Mảng hiển thị cập nhật chính xác: `pixel_data[0] = RED` và `pixel_data[1] = OFF`.

Kết quả mô phỏng:

Hình 12 thể hiện dạng sóng quá trình sinh và render đạn màu RED. Sau khi nhận xung `btn_event[0]`, module lập tức bật cờ `bullet_active = 1`, gán `bullet_position = 0` với mã màu RED. Mảng ngõ ra `pixel_data[0]` nhận giá trị màu tương ứng ngay lập tức, trong khi LED 1 liên tục duy trì trạng thái tắt (OFF).

Kết luận: Test Case 04 đạt yêu cầu (PASS).



Hình 12: Dạng sóng khởi tạo và hiển thị đàn màu RED tại LED 0

5.4.5 Test Case 05: Động học Di chuyển của Đạn (Bullet Movement)

Mục tiêu: Kiểm tra phản hồi của đạn khi xuất hiện xung định thời `bullet_tick`, đảm bảo tọa độ `bullet_position` tịnh tiến chính xác dọc theo dải LED về phía chỉ số cao hơn và vị trí hiển thị trên mảng `pixel_data` được cập nhật đồng thời.

Điều kiện kích thích:

- Dạng màu RED đang ở trạng thái kích hoạt tại vị trí khởi tạo (`bullet_active = 1`, `bullet_position = 0`).
- Kích hoạt một xung `bullet_tick = 1` kéo dài đúng 1 chu kỳ clock tại sườn âm của xung nhịp.
- Tín hiệu `snake_tick = 0` và không có sự kiện nút nhấn mới (`btn_event = 4'b0000`).

Kết quả mong đợi:

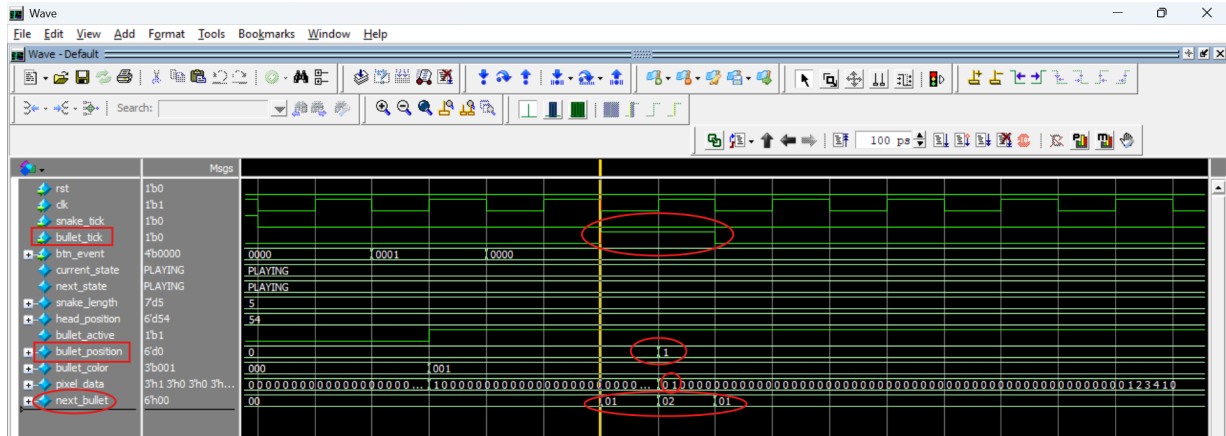
- Tọa độ đạn `bullet_position` tăng đúng 1 đơn vị (từ 0 lên 1) tại cạnh lên xung nhịp kế tiếp.
- Trạng thái kích hoạt và màu sắc đạn được duy trì ổn định (`bullet_active = 1`, `bullet_color = RED`).
- Mảng hiển thị LED dịch chuyển vị trí sáng tương ứng: `pixel_data[0]` tắt (OFF) và `pixel_data[1]` chuyển sang màu RED.

Kết quả mô phỏng:

Hình 13 thể hiện dạng sóng di chuyển của đàn dưới tác động của `bullet_tick`. Khi xung kích

hoạt xuất hiện, bộ tính toán look-ahead gán giá trị dự đoán `next_bullet = 1`. Tại cạnh lên tiếp theo của `clk`, thanh ghi `bullet_position` cập nhật thành công lên 1, đồng thời vị trí hiển thị trên `pixel_data` chuyển ngay từ LED 0 sang LED 1 mà không làm thay đổi trạng thái kích hoạt hay mã màu của đạn.

Kết luận: Test Case 05 đạt yêu cầu (PASS).



Hình 13: Dạng sóng kiểm tra sự tịnh tiến của đạn từ LED 0 sang LED 1 theo `bullet_tick`

5.4.6 Test Case 06: Va chạm Cùng Màu (Same-color Collision)

Mục tiêu: Kiểm tra phản hồi của hệ thống khi đạn va chạm với đầu rắn và có cùng màu sắc. Đốt đầu rắn tương ứng phải bị tiêu biến, chiều dài rắn giảm đi 1 đơn vị, đạn bị thu hồi về trạng thái không tích cực và FSM vẫn duy trì trạng thái `PLAYING` nếu rắn chưa bị diệt hết.

Điều kiện kích thích:

- FSM đang ở trạng thái `PLAYING`.
- Đầu rắn đang có màu `RED` và tọa độ xác định (`head_position`).
- Đạn màu `RED` đang hoạt động (`bullet_active = 1`, `bullet_color = RED`).
- Đạn được dịch chuyển đến vị trí chạm trán với đầu rắn thông qua xung tick.

Kết quả mong đợi:

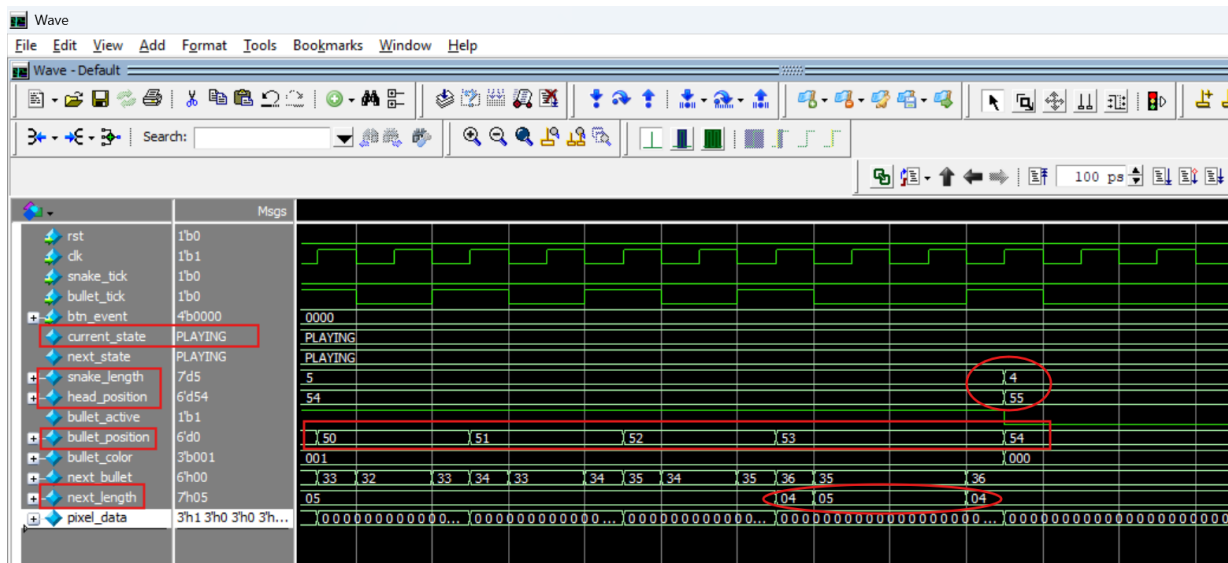
- Điều kiện va chạm được kích hoạt thành công.
- Chiều dài thân rắn `snake_length` giảm đi 1 đơn vị (từ 5 về 4).
- Vị trí đầu rắn cũ bị xóa bỏ khỏi mảng hiển thị.
- Cờ `bullet_active` chuyển từ 1 về 0 (đạn biến mất sau va chạm).
- Trạng thái FSM tiếp tục duy trì ở `PLAYING`.

Kết quả mô phỏng:

Hình 14 thể hiện dạng sóng quá trình xử lý va chạm trúng khi đạn và đầu rắn trùng màu. Khi đạn tiếp cận vị trí của đầu rắn, bộ giải mã va chạm xác nhận sự kiện trùng màu. Tại chu kỳ cập nhật tiếp theo, thanh ghi `snake_length` giảm tức thì từ 5 xuống 4, đồng thời đốt đầu cũ bị gỡ bỏ khỏi mảng logic. Đạn được vô hiệu hóa an toàn bằng cách hạ cờ `bullet_active` về 0.

mà không gây xung nhiễu lên các vị trí LED khác.

Kết luận: Test Case 06 đạt yêu cầu (PASS).



Hình 14: Dạng sóng kiểm tra va chạm cùng màu giữa đạn và đầu rắn

5.4.7 Test Case 07: Va chạm Khác Màu (Different-color Collision)

Mục tiêu: Kiểm tra khả năng phân biệt màu sắc khi xảy ra va chạm: nếu màu đạn không trùng khớp với màu của đầu rắn, đạn phải bị thu hồi ngay lập tức, trong khi thân rắn, chiều dài và màu sắc các đốt của rắn được giữ nguyên vẹn.

Điều kiện kích thích:

- FSM đang ở trạng thái PLAYING.
- Đầu rắn hiện tại mang mã màu RED (3'b001).
- Kích hoạt một viên đạn mang màu YELLOW (bullet_color = 3'b100) đến vị trí chạm trán với đầu rắn.
- Bơm các xung bullet_tick tuần tự để đẩy đạn tới sát đầu rắn, sau đó kích hoạt 1 xung bullet_tick cuối cùng để tạo va chạm trúng màu.

Kết quả mong đợi:

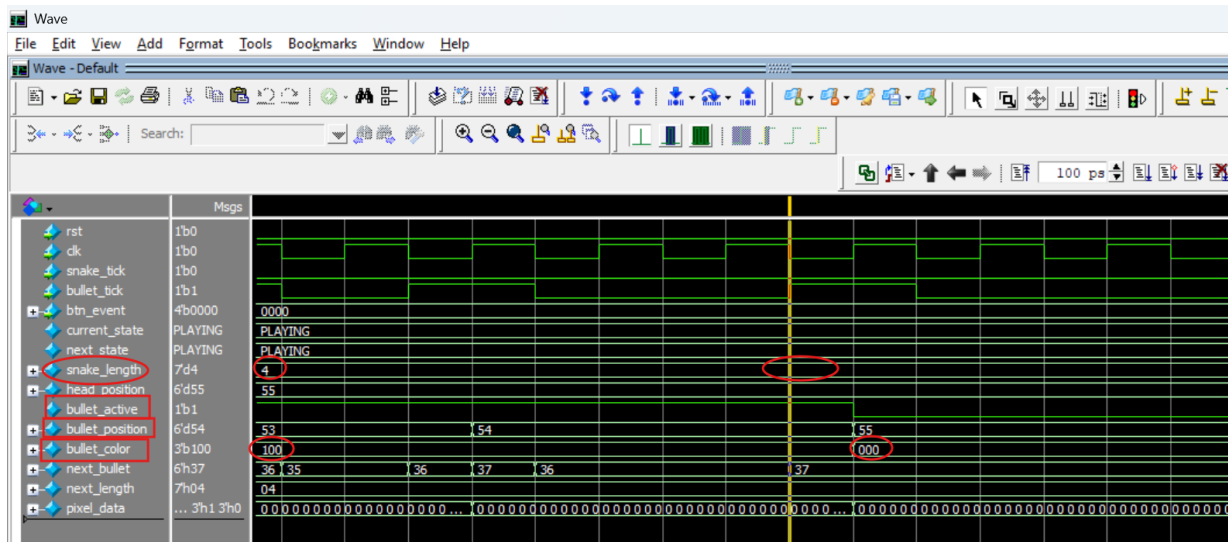
- Mạch phát hiện va chạm xác lập sự kiện nhưng nhận diện kết quả sai màu (mismatch).
- Chiều dài thân rắn snake_length giữ nguyên không đổi.
- Mảng thân rắn snake_array và vị trí đầu rắn giữ nguyên các giá trị ban đầu.
- Tín hiệu bullet_active chuyển về mức 0 (đạn bị vô hiệu hóa).
- Trạng thái FSM tiếp tục duy trì ở PLAYING.

Kết quả mô phỏng:

Hình 15 ghi nhận dạng sóng phản hồi khi đạn va chạm sai màu với đầu rắn. Khi đạn YELLOW tiếp xúc với đầu rắn RED, bộ giải mã logic nhận diện sự bất đồng bộ về màu sắc. Tại cạnh lên

xung nhịp cập nhật trạng thái, cờ `bullet_active` được kéo về 0 để giải phóng làn đạn, đồng thời thanh ghi `snake_length` không hề bị suy giảm và vị trí hiển thị của rắn trên `pixel_data` được bảo toàn nguyên vẹn.

Kết luận: Test Case 07 đạt yêu cầu (PASS).



Hình 15: Dạng sóng kiểm tra va chạm khác màu giữa đạn và đầu rắn

5.4.8 Test Case 08: Kích hoạt Điều kiện Thắng (WIN Condition)

Mục tiêu: Kiểm tra khả năng nhận diện điều kiện chiến thắng của FSM thông qua quy trình bắn hạ liên tục toàn bộ các đốt thân rắn cho đến khi chiều dài rắn giảm về 0 (`snake_length = 0`).

Điều kiện kích thích:

- Hệ thống đang chạy ở trạng thái **PLAYING**.
- Sử dụng vòng lặp tự động trong testbench: đọc mã màu tại đầu rắn `snake_array[head_position]` và phát xung `btn_event` tạo đạn có màu tương ứng.
- Bơm các xung `bullet_tick` tuần tự để đẩy đạn tới sát đầu rắn, sau đó kích hoạt 1 xung `bullet_tick` cuối cùng để tạo va chạm trúng màu.
- Lặp lại quy trình triệt tiêu trên cho đến khi toàn bộ các đốt rắn bị loại bỏ.

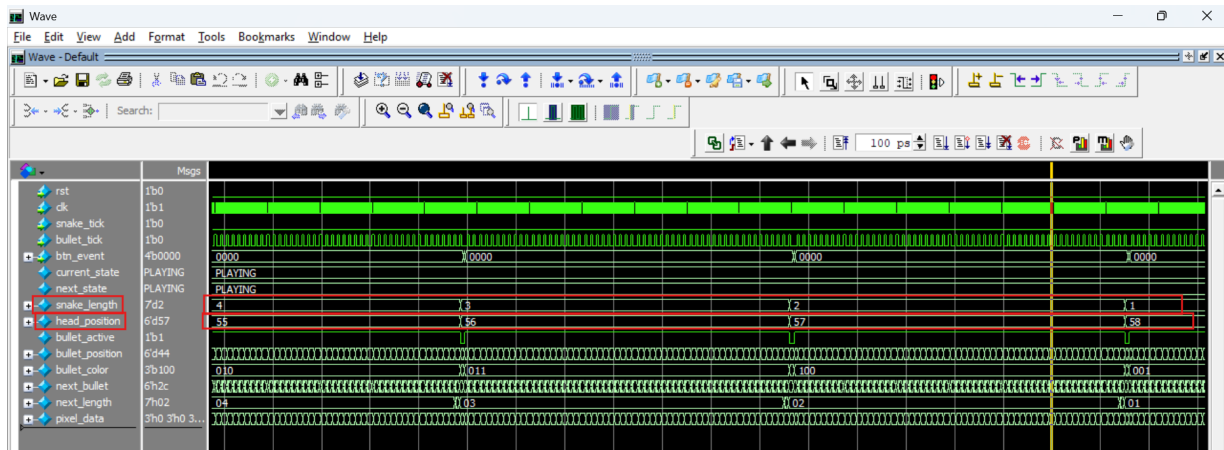
Kết quả mong đợi:

- Độ dài thân rắn giảm tuần tự sau mỗi lần bắn trúng và đạt `snake_length = 0`.
- FSM lập tức chuyển từ trạng thái **PLAYING** sang trạng thái **WIN** (`current_state == WIN`).
- Không còn viên đạn nào ở trạng thái kích hoạt sau khi rắn bị tiêu diệt hết.

Kết quả mô phỏng:

Hình 16 và Hình 17 minh họa toàn bộ tiến trình tự động bắn hạ thân rắn và chuyển đổi trạng thái FSM. Dưới sự điều phối của testbench, từng đốt rắn bị triệt tiêu tương ứng sau mỗi pha va chạm cùng màu. Sau khi `snake_length` giảm về 0, FSM chuyển sang trạng thái **WIN** ở chu kỳ cập nhật kế tiếp.

Kết luận: Test Case 08 đạt yêu cầu (PASS).

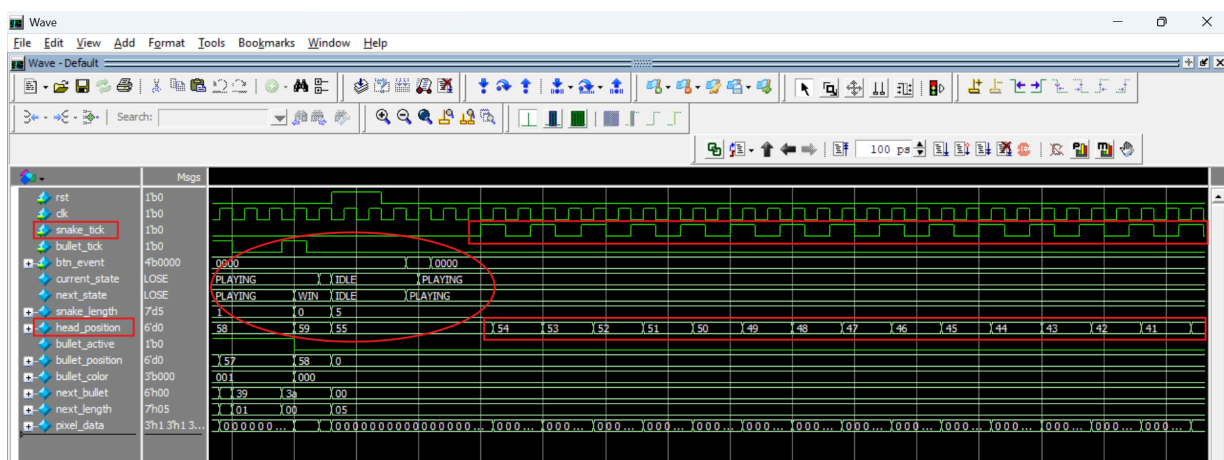


- Tọa độ đầu rắn chạm biên LED 0: `head_position == 0`.
- FSM chuyển trạng thái tức thì từ `PLAYING` sang `LOSE` (`current_state == LOSE`).
- Hệ thống khóa điều khiển, không cho phép đầu rắn tiếp tục giảm xuống dưới 0.

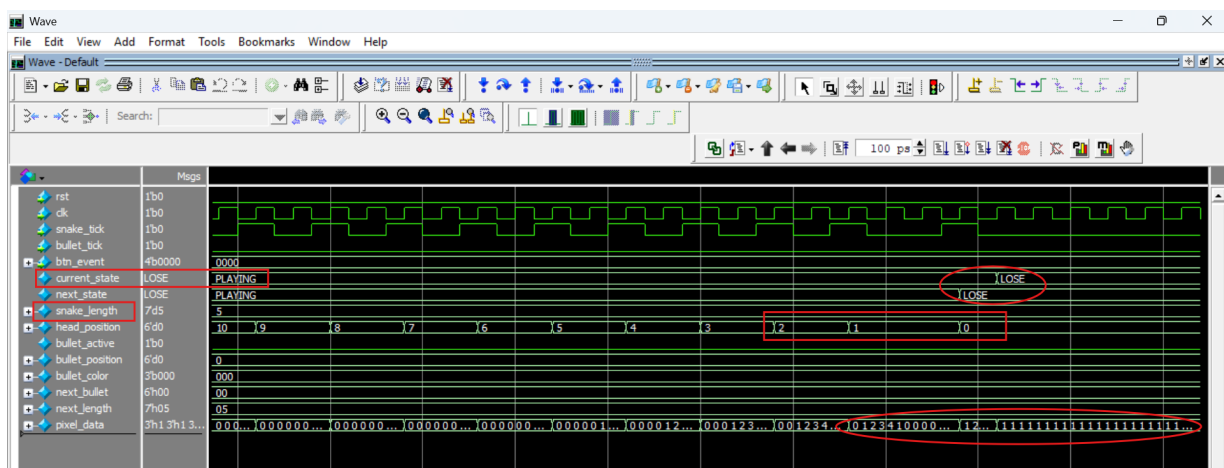
Kết quả mô phỏng:

Hình 18 và Hình 19 minh họa tiến trình tịnh tiến của thân rắn về phía LED 0 và thời điểm kích hoạt trạng thái `LOSE`. Sau khi hoàn tất reset và chuyển sang `PLAYING`, các xung `snake_tick` đẩy dần `head_position` từ 55 về 1. Ngay tại cạnh lên xung nhịp nhận xung tick cuối cùng, `head_position` đạt giá trị 0, đồng thời biến trạng thái `current_state` chuyển đổi sang `LOSE` ở clock kế tiếp, đáp ứng đúng tiêu chí đánh giá kiểm thử.

Kết luận: Test Case 09 đạt yêu cầu (PASS).



Hình 18: Dạng sóng quá trình tịnh tiến liên tục của rắn về phía LED 0



Hình 19: Dạng sóng head_position chạm LED 0 và FSM chuyển sang LOSE

5.4.10 Test Case 10: Khởi động lại từ Trạng thái Thua (Restart from LOSE)

Mục tiêu: Kiểm tra khả năng khôi phục toàn diện của hệ thống khi nhận tín hiệu reset từ trạng thái kết thúc `LOSE`, đảm bảo FSM quay về `IDLE`, các thông số vị trí và độ dài thân rắn được nạp lại cấu hình mặc định ban đầu, đồng thời xóa bỏ hoàn toàn trạng thái đạn.

Điều kiện kích thích:

- Hệ thống đang bị đóng băng tại trạng thái kết thúc LOSE từ TC09.
- Tác động xung reset tích cực mức cao `rst = 1` trong 2 chu kỳ xung nhịp tại sườn âm của clock.
- Hạ `rst` về 0 và quan sát sự tái lập trạng thái của module.

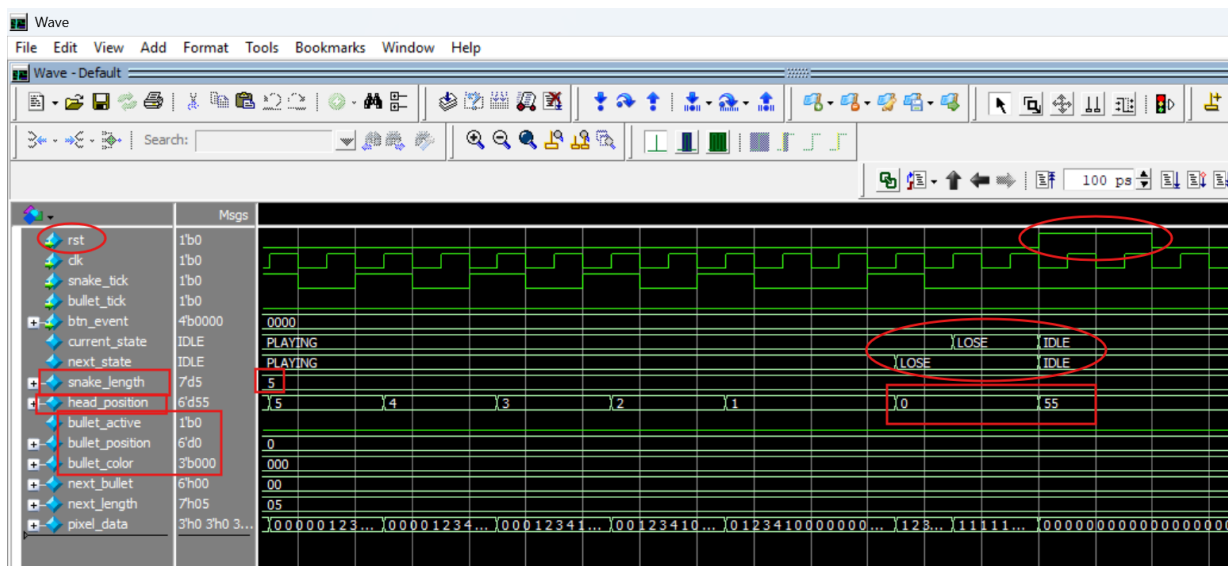
Kết quả mong đợi:

- FSM thoát khỏi LOSE và quay về trạng thái an toàn IDLE (`current_state == IDLE`).
- Chiều dài thân rắn khôi phục về giá trị mặc định: `snake_length == LENGTH_TEST` (5 segments).
- Tọa độ đầu rắn được đặt lại tại vị trí xuất phát: `head_position == LED_TEST - LENGTH_TEST` (LED 55).
- Trạng thái đạn được xóa bỏ hoàn toàn (`bullet_active == 0`).

Kết quả mô phỏng:

Hình 20 thể hiện dạng sóng quá trình giải phóng trạng thái khóa và tái thiết lập hệ thống từ trạng thái LOSE. Ngay khi `rst` được kích hoạt, máy trạng thái FSM lập tức chuyển từ LOSE về IDLE. Các thanh ghi điều khiển được nạp lại đầy đủ các thông số khởi tạo chuẩn (`snake_length = 5`, `head_position = 55`), đồng thời cờ `bullet_active` được xóa về 0, đưa hệ thống sẵn sàng cho lượt chơi mới.

Kết luận: Test Case 10 đạt yêu cầu (PASS).



Hình 20: Dạng sóng khôi phục hệ thống về IDLE từ trạng thái LOSE sau khi Reset

5.4.11 Test Case 11: Xung Kích hoạt Đồng thời (Simultaneous Ticks)

Mục tiêu: Kiểm tra tính toàn vẹn của logic cập nhật trạng thái khi cả hai xung `bullet_tick` và `snake_tick` cùng tích cực trong một chu kỳ xung nhịp. Đánh giá hai giai đoạn: chuyển động tịnh tiến đồng thời và va chạm trực diện trong cùng một chu kỳ tick kép (kiểm tra cơ chế Anti-Tunneling).

Điều kiện kích thích:

- Chuyển FSM sang PLAYING và tạo một viên đạn màu RED tại LED 0 (`bullet_active = 1`).
- **Giai đoạn 1 (Chuyển động đồng thời):** Kích hoạt đồng thời `bullet_tick = 1` và `snake_tick = 1` trong 1 chu kỳ clock.
- Bơm các xung `bullet_tick` đơn để đẩy đạn đến sát đầu rắn (`bullet_position < head_position - 1`).
- **Giai đoạn 2 (Va chạm tick kép):** Kích hoạt đồng thời `bullet_tick = 1` và `snake_tick = 1` ngay tại thời điểm đạn và đầu rắn đối đầu trực tiếp.

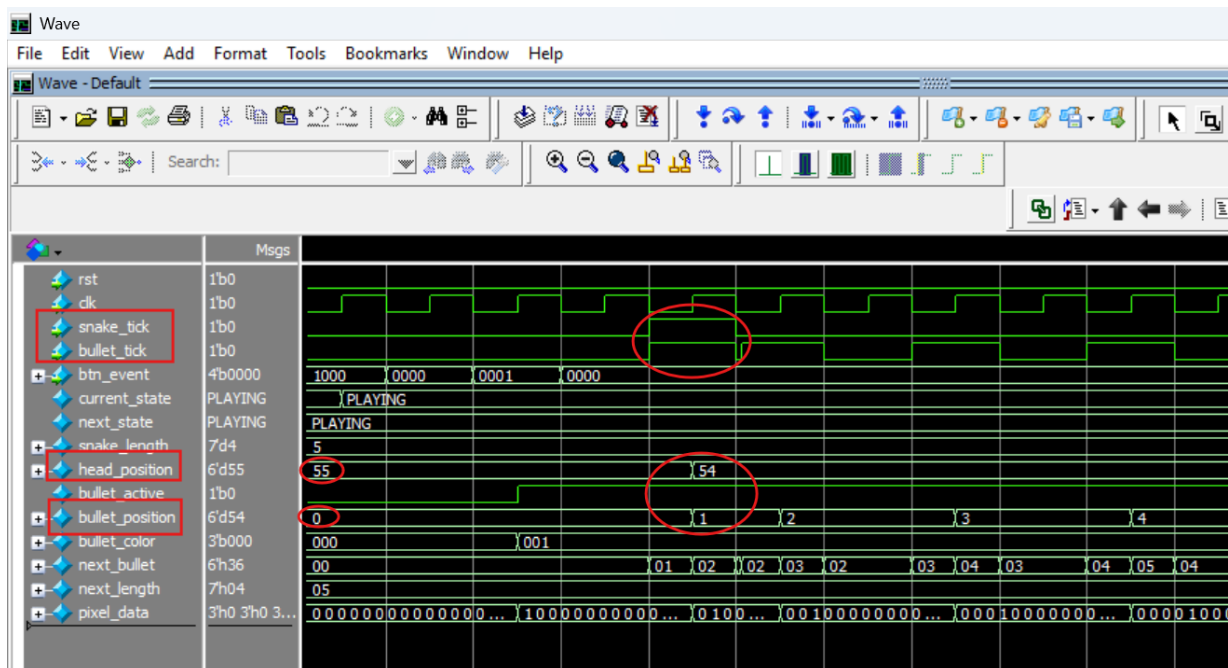
Kết quả mong đợi:

- **Tại Giai đoạn 1:** Đạn tịnh tiến lên LED 1; rắn tịnh tiến về dải LED 54–58, LED 59 tắt (OFF); mảng `pixel_data` render đồng thời cả đạn và thân rắn chính xác.
- **Tại Giai đoạn 2:** Va chạm được phát hiện tức thì; `bullet_active` về 0, mã màu đạn chuyển về OFF; độ dài `snake_length` giảm đi 1; vị trí đầu `head_position` dời sang LED 55 (đốt đầu đỏ tại LED 54 bị xóa); mảng LED cập nhật chính xác với LED 53, 54 tắt và các đốt còn lại hiển thị GREEN (55), BLUE (56), YELLOW (57), RED (58).

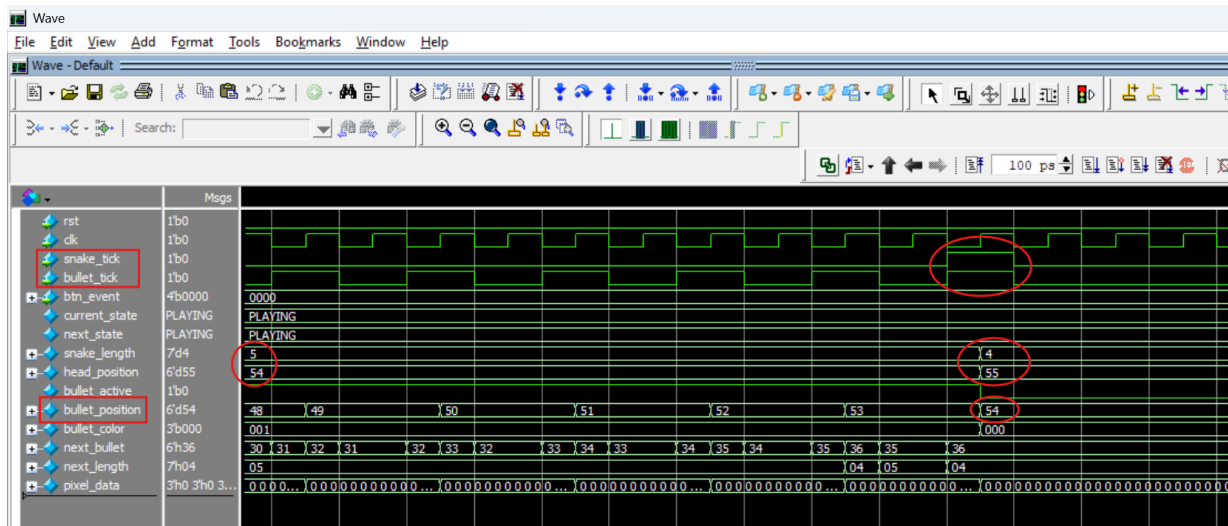
Kết quả mô phỏng:

Hình 21 và Hình 22 minh họa chi tiết hai pha kiểm thử của Test Case 11. Ở pha đầu tiên, đạn và thân rắn đều dịch chuyển đồng thời 1 vị trí mà không gây xung đột tín hiệu. Ở pha va chạm, dù cả đạn và đầu rắn cùng tịnh tiến đối đầu trong cùng một chu kỳ clock, mạch Look-ahead đã giải quyết va chạm chính xác: đốt đầu đỏ tại LED 54 bị triệt tiêu, đạn biến mất và đầu rắn mới dời về vị trí 55 mà không xảy ra hiện tượng đạn bị xuyên qua thân rắn.

Kết luận: Test Case 11 đạt yêu cầu (PASS).



Hình 21: Dạng sóng dịch chuyển đồng thời của đạn và rắn khi kích hoạt tick kép



Hình 22: Dạng sóng xử lý va chạm trúng cùng màu trong cùng chu kỳ kích hoạt tick kép

5.4.12 Test Case 12: Chống Bắn Đạn Dồn Dập (Rapid Fire Prevention Stress Test)

Mục tiêu: Kiểm tra khả năng bảo vệ hệ thống trước các thao tác nhấn phím bất thường khi đang có một viên đạn bay trên dải LED (`bullet_active = 1`): nhấn ngẫu nhiên liên tục, nhấn tổ hợp nhiều phím cùng lúc, và giữ đè phím trong nhiều chu kỳ xung nhịp. Đảm bảo viên đạn đang bay không bị biến dạng về màu sắc hay vị trí, và không có viên đạn mới nào bị sinh đề lên vị trí LED 0.

Điều kiện kích thích:

- Bắn một viên đạn màu GREEN (`bullet_color = 3'b010`) và cấp 3 xung `bullet_tick` để đẩy đạn tới vị trí LED 3 (`bullet_position = 3`).
- **Pha 1 (Spam ngẫu nhiên):** Kích hoạt xung phím ngẫu nhiên `4'b0001` « `$urandom_range(0,3)` liên tiếp trong 3 chu kỳ clock.
- **Pha 2 (Tổ hợp phím):** Kích hoạt đồng thời toàn bộ các phím (`btn_event = 4'b1111`) trong 1 chu kỳ clock.
- **Pha 3 (Giữ đè phím):** Giữ đè phím RED (`btn_event = 4'b0001`) liên tục trong suốt 5 chu kỳ clock.
- **Pha 4 (Kiểm tra LED 0):** Quan sát giá trị xuất ra của `pixel_data[0]` trong suốt các pha tác động trên.

Kết quả mong đợi:

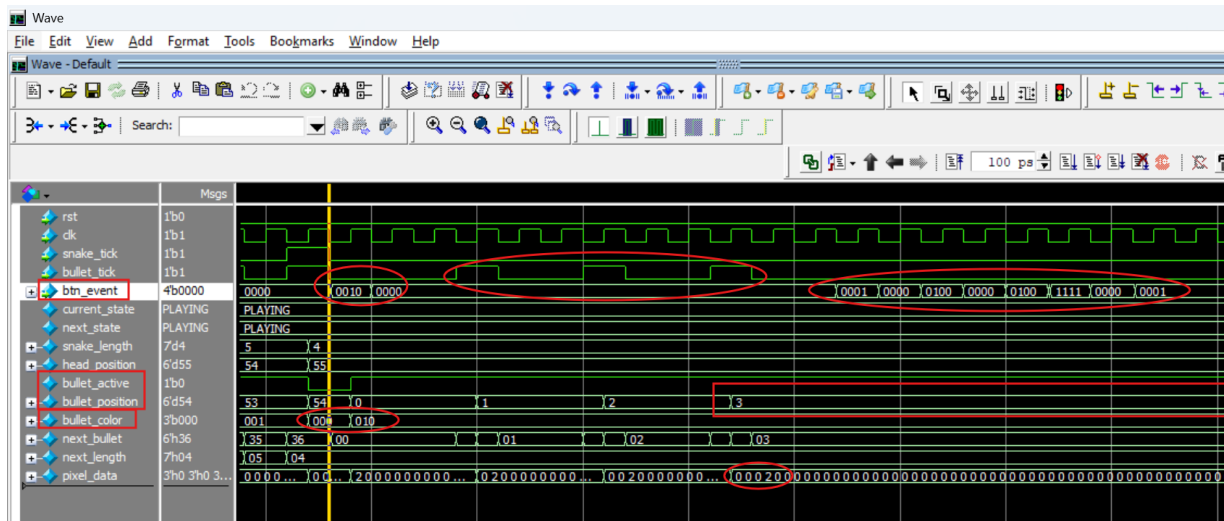
- Viên đạn GREEN đang bay hoàn toàn miễn nhiễm với mọi kích thích nút bấm: `bullet_color` giữ nguyên màu GREEN. Trong điều kiện không có `bullet_tick`, các `btn_event` bổ sung không làm thay đổi `bullet_position`, `bullet_color` hoặc `bullet_active`.
- Không có bất kỳ viên đạn mới nào được nạp vào mảng hiển thị: `pixel_data[0] == OFF` (LED 0 luôn giữ trạng thái tắt).

Kết quả mô phỏng:

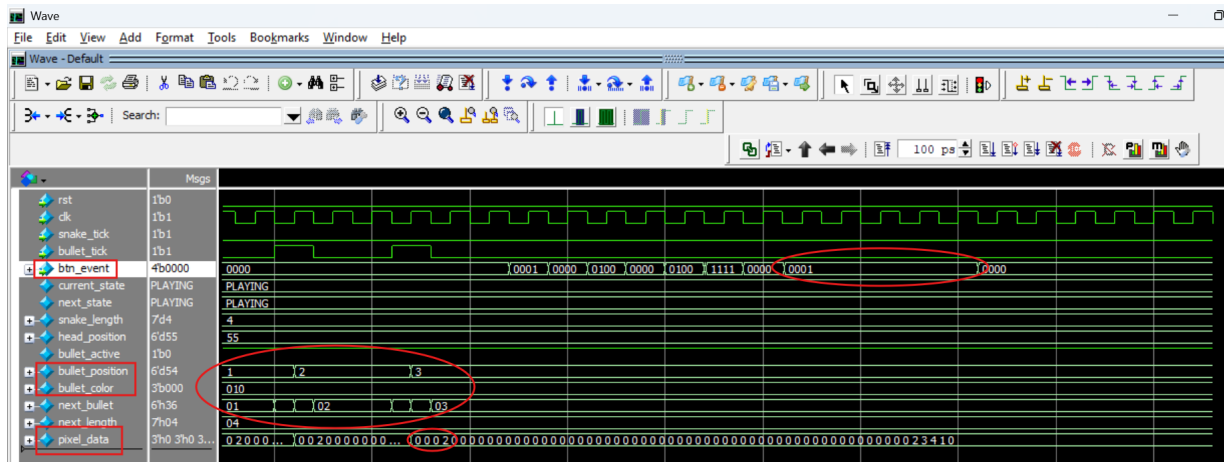
Hình 23 và Hình 24 ghi nhận phản hồi của module trong các pha kiểm thử ứng suất nút nhấn.

Khi một viên đạn GREEN đã được bắn ra và di chuyển đến vị trí LED 3, mọi hành vi spam ngẫu nhiên, bấm tổ hợp nhiều phím cùng lúc, hay giữ đè phím RED đều bị logic điều khiển chặn lại an toàn. Vị trí và mã màu của viên đạn đang bay duy trì ổn định tuyệt đối. Đồng thời, `pixel_data[0]` giữ mức OFF xuyên suốt quá trình thử nghiệm, chứng minh cơ chế chỉ cho phép duy nhất một viên đạn hoạt động trên dải LED hoạt động chính xác và tin cậy.

Kết luận: Test Case 12 đạt yêu cầu (PASS).



Hình 23: Dạng sóng kiểm tra tính cô lập của đạn khi spam nút ngẫu nhiên và bấm tổ hợp phím



Hình 24: Dạng sóng kiểm tra giữ đè phím và xác nhận không sinh đạn đè tại LED 0

6 Kết quả Mô phỏng Tích hợp Toàn hệ thống (Top-Level Verification)

6.1 Mục tiêu Kiểm thử

Kiểm tra khả năng tích hợp toàn diện giữa các khối chức năng (`button_conditioner`, `tick_generator`, `game_engine`, và `ws2812b_driver`) trên module đỉnh `top_snake_game`.

Mục tiêu chính bao gồm:

- Xác minh tính đồng bộ của tín hiệu reset toàn hệ thống từ ngõ vào `rst_sw`.
- Đảm bảo đường truyền điều khiển xuyên suốt: từ sự kiện nhấn phím vật lý (`key[3:0]`), qua bộ lọc xung, kích hoạt tạo đạn trên `game_engine`, cập nhật mảng `pixel_data` và truyền ra chân vật lý `led_data_out`.
- Xác minh khả năng bắt nhịp tự động giữa bộ tạo thời gian `tick_generator` và logic di chuyển của trò chơi.
- Đảm bảo giao thức vật lý đơn dây 1-wire tại chân `led_data_out` tuân thủ chặt chẽ định thời bit chuẩn WS2812B thông qua việc đo đặc thời gian thực và giám sát liên tục bằng SystemVerilog Assertions (SVA).

6.2 Quy trình Kiểm thử Đầu-cuối (End-to-End Verification Flow)

Để tối ưu thời gian mô phỏng mà vẫn giữ nguyên bản chất hoạt động của mạch, testbench ghi đè các tham số phần cứng: `BULLET_CYCLES = 20` chu kỳ clock (thay vì 2.500.000 chu kỳ), `RES = 40` chu kỳ (thay vì 15.000 chu kỳ), và kiểm thử trên quy mô rút gọn `LED_COUNT = 6`.

Quy trình kiểm thử đầu-cuối thực thi tuần tự qua các giai đoạn:

1. **Khởi tạo và Reset toàn hệ thống (TC1):** Kích hoạt xung `rst_sw = 1`. Toàn bộ các module con được giám sát để xác nhận FSM chuyển về trạng thái ban đầu (`game_engine` về IDLE, `ws2812b_driver` về LOAD).
2. **Chuyển trạng thái Vận hành (TC2):** Nhấn phím `key[2]` (active-low) để đưa toàn hệ thống từ IDLE sang PLAYING.
3. **Luồng sự kiện xuyên suốt (TC3 & TC4):** Nhấn `key[0]` để phát đạn RED. Testbench giám sát chuỗi phản hồi từ `key[0] → sinh xung btn_event[0] → game_engine` bật cờ `bullet_active = 1`, gán màu RED → mảng `pixel_data[0]` nhận mã RED. Đồng thời kiểm tra đúng ánh xạ dây (wiring) cho các phím còn lại (`key[1]`, `key[3]`).
4. **Đồng bộ nhịp Tick nội bộ (TC5):** Đợi qua chu kỳ `BULLET_CYCLES_TEST` xung nhịp mà không can thiệp từ testbench để xác nhận `tick_generator` phát xung nhịp tự nhiên làm tăng tọa độ `bullet_position`.
5. **Đo kiểm dạng sóng ngõ ra thực tế (TC6):** Sử dụng task tự động `read_pixel_24bit` đóng vai trò như một bộ thu WS2812B ảo, đo trực tiếp thời gian mức cao T_H và mức thấp T_L của từng bit trên đường truyền `led_data_out`. Dữ liệu 24-bit GRB giải mã được kiểm tra khớp với mã màu RED chuẩn (24'h00_3F_00).

6.3 Bằng chứng Mô phỏng Dạng sóng (Waveform Evidence)

6.3.1 Hành vi Reset, Khởi động và Kích hoạt Đạn xuyên suốt

- **Điều kiện kích thích:** Tác động `rst_sw = 1`, sau đó lần lượt nhấn xung `key[2]` để vào `PLAYING` và nhấn `key[0]` để bắn đạn `RED`.
- **Kết quả mô phỏng:**
 - Toàn bộ FSM của hệ thống được tái lập đồng bộ về trạng thái mặc định.
 - Sườn xuống của `key[0]` được bộ lọc bắt chính xác và sinh ra đúng 1 xung `btn_event[0]` tại sườn lên xung nhịp kế tiếp (thỏa mãn SVA `p_btn0_event`).
 - Cờ `bullet_active` bật lên 1 và `pixel_data[0]` nạp thành công mã màu `RED`, chứng minh kết nối liên module thông suốt.

Hình 25: Dạng sóng kiểm tra Reset, chuyển trạng thái `PLAYING` và đường truyền nút nhấn xuyên suốt hệ thống

6.3.2 Tĩnh tiến Tự động và Kiểm tra Giao thức PWM tại `led_data_out`

- **Điều kiện kích thích:** Để mạch tự chạy dưới sự điều phối của `tick_generator` và thu nhận chuỗi xung bit nối tiếp tại chân `led_data_out`.
- **Kết quả mô phỏng:**
 - Xung `bullet_tick` từ `tick_generator` kích hoạt đúng chu kỳ, làm đạn tĩnh tiến tự động sang vị trí kế tiếp mà không cần can thiệp ngoại vi.
 - Tín hiệu nối tiếp trên chân `led_data_out` chuyển đổi trạng thái liên tục, không xuất hiện trạng thái treo mức cao quá 45 chu kỳ xung nhịp (thỏa mãn SVA `p_no_stuck_high`).
 - Dạng sóng PWM đạt đúng các thông số: Bit 0 ($T_H \approx 300$ ns, $T_L \approx 800$ ns) và Bit 1 ($T_H \approx 800$ ns, $T_L \approx 300$ ns), giải mã thành công 24-bit GRB của Pixel 0.

Hình 26: Dạng sóng tĩnh tiến đạn theo xung tick nội bộ và chuỗi xung PWM thực tế tại chân `led_data_out`

7 Tổng kết Kết quả Kiểm thử

7.1 Tổng hợp Kết quả Kiểm thử

Bảng 3: Tổng hợp kết quả kiểm thử các khối chức năng trong toàn bộ hệ thống

Khối chức năng	Phạm vi kiểm thử (Verification Scope)	Số TC	Kết quả
ws2812b_driver	FSM, tuần tự hóa dữ liệu, định thời xung PWM, chu kỳ chốt Latch, cô lập khung hình	7	PASS
game_engine	FSM, động học rắn/đạn, phân giải va chạm, điều kiện WIN/LOSE, look-ahead anti-tunneling, chống spam phím	12	PASS
button_conditioner	Đồng bộ xung nhịp, chống rung cơ học (de-bounce), bắt cạnh xung	—	<i>Not included</i>
tick_generator	Phân tần số, tạo xung enable đơn chu kỳ 50 ms và 250 ms	—	<i>Not included</i>
top_snake_game	Tích hợp liên module, kết nối tín hiệu điều khiển và bus dữ liệu toàn hệ thống	1	PASS

7.2 Nhận xét Đánh giá

Dựa trên kết quả mô phỏng chức năng trên môi trường QuestaSim, toàn bộ các khối xử lý trung tâm và ngoại vi giao tiếp của hệ thống đều đáp ứng đầy đủ các tiêu chuẩn kỹ thuật đề ra trong tài liệu đặc tả:

- Khối **ws2812b_driver** đã được chứng minh đáp ứng chặt chẽ các yêu cầu vật lý khắt khe về định thời xung số NZR, thực thi đúng quy trình tuần tự hóa dữ liệu MSB-first và bảo vệ toàn vẹn khung hình đang phát nhờ cơ chế chốt snapshot độc lập.
- Khối **game_engine** thể hiện khả năng điều phối chính xác máy trạng thái FSM, tính toán động lực học di chuyển mượt mà theo từng nhịp tick, xử lý linh hoạt và chính xác các va chạm logic bằng kỹ thuật look-ahead mà không gặp hiện tượng tranh chấp hay xuyên đạn, đồng thời bảo đảm tính toàn vẹn của dữ liệu hiển thị **pixel_data**.

8 Kết luận và Hướng Phát triển

8.1 Những Kết quả Đạt được

Đề tài đã hoàn thành các mục tiêu đề ra trong giai đoạn thiết kế và kiểm thử hệ thống phần cứng:

- Hoàn thiện kiến trúc vi mạch số theo chuẩn thiết kế đồng bộ đơn miền xung nhịp 50 MHz sử dụng ngôn ngữ SystemVerilog.
- Hiện thực hóa đầy đủ các khối chức năng theo cấu trúc phân tầng rõ ràng giữa khối điều khiển (Control Flow) và đường dẫn dữ liệu (Data Path).

- Xây dựng môi trường kiểm thử tự động (Self-checking Testbench) với các task kích thích và assertion kiểm tra độc lập cho từng module.
- Xác minh thành công các hành vi chức năng trọng yếu và các trường hợp biên nguy hiểm thông qua phân tích dạng sóng mô phỏng chi tiết.
- Đóng gói và tích hợp thành công mô hình toàn hệ thống ở mức Top-level.

8.2 Giới hạn của Báo cáo Kiểm thử

Bên cạnh các kết quả đạt được, báo cáo kiểm thử này có một số giới hạn phạm vi như sau:

- Các module `button_conditioner` và `tick_generator` không được trình bày dưới dạng chương mục báo cáo chi tiết riêng biệt do quy mô logic tương đối đơn giản và mang tính chất mạch đếm kinh điển; các module này đã được kiểm tra qua mô phỏng chức năng cơ bản trước khi tích hợp vào khối Top.
- Toàn bộ các kiểm thử mới dừng lại ở mức mô phỏng trước tổng hợp (Pre-synthesis Functional Simulation / RTL Simulation), chưa bao gồm kiểm thử trễ định thời thực tế (Gate-level / Post-fit Simulation with SDF timing back-annotation).

8.3 Hướng Phát triển Tiếp theo

Để hoàn thiện hệ thống hướng tới triển khai sản phẩm thương mại hoặc các thiết kế vi mạch chuyên sâu hơn, đề tài có thể tiếp tục mở rộng theo các hướng:

- Mở rộng bộ Testbench tự động sang mô hình hướng đối tượng với phương pháp luận UVM (Universal Verification Methodology), tích hợp thêm Function Coverage và Code Coverage để đo lường độ phủ kiểm thử.
- Bổ sung hệ thống SystemVerilog Assertions (SVA) tĩnh và động vào mã nguồn RTL để bắt lỗi giao tiếp bus và lỗi định thời ngay trong thời gian thực.
- Tiến hành nạp bitstream lên kit FPGA Altera DE2 vật lý (Cyclone II EP2C35F672C6), đo đặc dạng sóng thực tế trên chân GPIO bằng máy hiện sóng (Oscilloscope / Logic Analyzer) và đánh giá độ trễ nhiệt/định thời (Timing Closure).